

Exp: 5

STUDY OF ACTIVATION FUNCTIONS AND ITS ROLES

AIM:

To study different activation functions used in neural network and analyse their role.

ALGORITHM:

* Define common activation functions:

→ Sigmoid.

→ Hyperbolic Tangent (Tanh)

→ Rectified Linear Unit (ReLU)

→ leaky ReLU

→ Softmax.

* Generate a range of input values.

* Apply each activation function on the input.

* plot the outputs to study their characteristics

* Observe how each functions handles linearity, vanishing gradients and sparsity.

Equations:

* Sigmoid $\Rightarrow \frac{1}{1+e^x}$

* Tanh $\Rightarrow \frac{2}{1+e^{-2x}}$

* ReLU $\Rightarrow \max(0, x)$

PSEUDOCODE:

BEGIN

Define sigmoid $(x) = 1 / (1 + \exp(-x))$

Define tanh $(x) = (\exp(x) - \exp(-x)) / (\exp(x) + \exp(-x))$

Define relu $(x) = \max(0, x)$

Define leaky-relu $(x) = x$ if $x > 0$ else αx

Define softmax $(x) = \exp(x) / \sum \exp(x)$

create input values from -10 to 10

For each activation function;

Compute output values

plot function curve.

END FOR

END.

TESTING:

* For $n = 5$:

→ Sigmoid ≈ 0.9933

→ Tanh ≈ 0.9999

→ ReLU = 5

→ Leaky ReLU = 5

* For $n = [1, 2, 3]$

→ Softmax = $[0.09, 0.24, 0.66]$

RANGE:

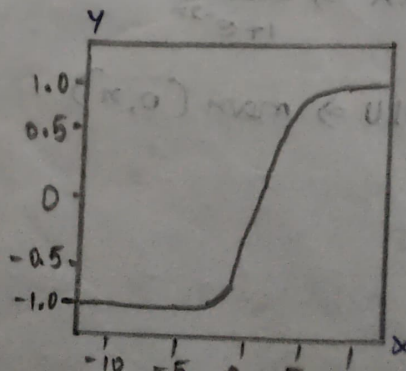
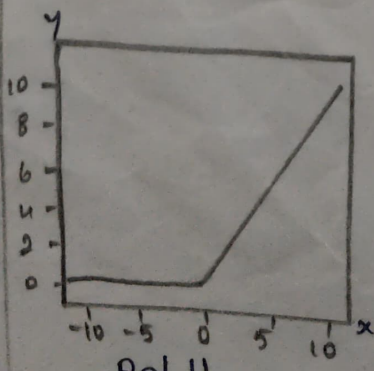
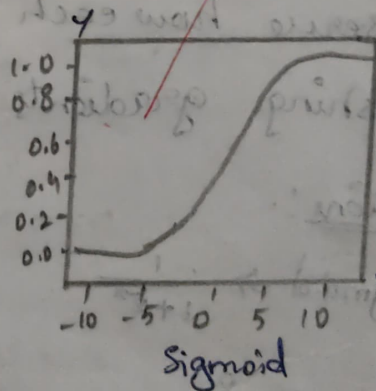
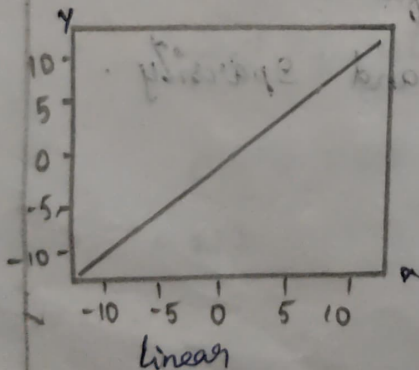
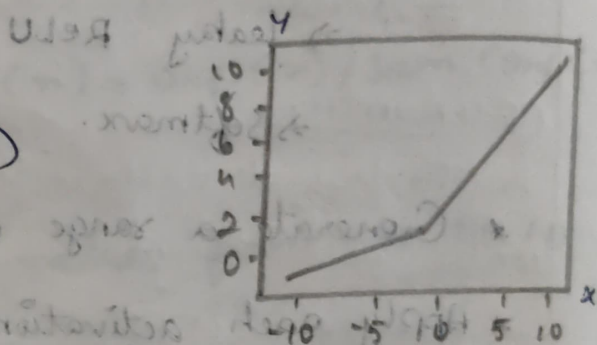
* Sigmoid - $(0, 1)$

* Tanh - $(-1, 1)$

* ReLU - $[0, \infty)$

* Leaky ReLU - $(-\infty, \infty)$

* Softmax - $(0, 1)$ sum to 1.



OBSERVATION:

- ⇒ Sigmoid maps inputs to range $(0,1)$, useful for probabilities but suffers from vanishing gradient for large values.
- ⇒ Tanh maps input to range $(-1,1)$.
- ⇒ ReLU outputs positive values directly and zero otherwise → efficient and widely used in deep networks.
- ⇒ Leaky ReLU allows small negative values, preventing the dying ReLU problem.
- ⇒ Softmax converts outputs into probability distribution, commonly used in the output layer for classification.

RESULT:

Therefore the study of activation functions and its roles are completed successfully.

```

import torch
import torch.nn as nn
import torch.nn.functional as F
import matplotlib.pyplot as plt
from sklearn.datasets import make_moons
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler

def plot_activation_functions():
    x = torch.linspace(-10, 10, 100)
    activations = {
        'Linear': x,
        'Sigmoid': torch.sigmoid(x),
        'Tanh': torch.tanh(x),
        'ReLU': F.relu(x),
        'Leaky ReLU': F.leaky_relu(x, negative_slope=0.1),
    }
    plt.figure(figsize=(12, 6))
    for i, (name, y) in enumerate(activations.items()):
        plt.subplot(2, 3, i + 1)
        plt.plot(x.numpy(), y.numpy())
        plt.title(name)
        plt.grid(True)
    plt.tight_layout()
    plt.show()

plot_activation_functions()

X, y = make_moons(n_samples=1000, noise=0.2, random_state=42)
X = StandardScaler().fit_transform(X)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42)

X_train = torch.tensor(X_train, dtype=torch.float32)
y_train = torch.tensor(y_train, dtype=torch.long)
X_test = torch.tensor(X_test, dtype=torch.float32)
y_test = torch.tensor(y_test, dtype=torch.long)

class NetReLU(nn.Module):
    def __init__(self):
        super().__init__()

```

```

    super().__init__()
    self.fc1 = nn.Linear(2, 16)
    self.fc2 = nn.Linear(16, 2)

    def forward(self, x):
        x = F.relu(self.fc1(x))
        return self.fc2(x)

class NetSigmoid(nn.Module):
    def __init__(self):
        super().__init__()
        self.fc1 = nn.Linear(2, 16)
        self.fc2 = nn.Linear(16, 2)

    def forward(self, x):
        x = torch.sigmoid(self.fc1(x))
        return self.fc2(x)

class NetTanh(nn.Module):
    def __init__(self):
        super().__init__()
        self.fc1 = nn.Linear(2, 16)
        self.fc2 = nn.Linear(16, 2)

    def forward(self, x):
        x = torch.tanh(self.fc1(x))
        return self.fc2(x)

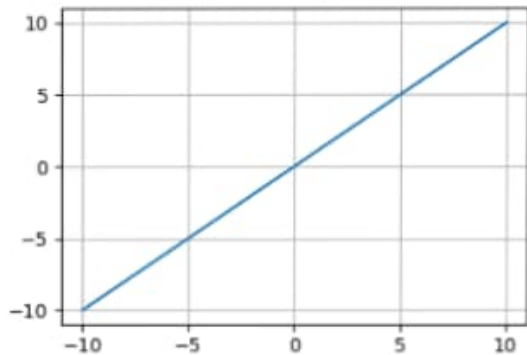
def train_model(model, name):
    optimizer = torch.optim.Adam(model.parameters(), lr=0.01)
    criterion = nn.CrossEntropyLoss()
    for epoch in range(100):
        optimizer.zero_grad()
        out = model(X_train)
        loss = criterion(out, y_train)
        loss.backward()
        optimizer.step()

    acc = (model(X_test).argmax(dim=1) == y_test).float().mean()
    print(f'{name} Test Accuracy: {acc:.4f}')

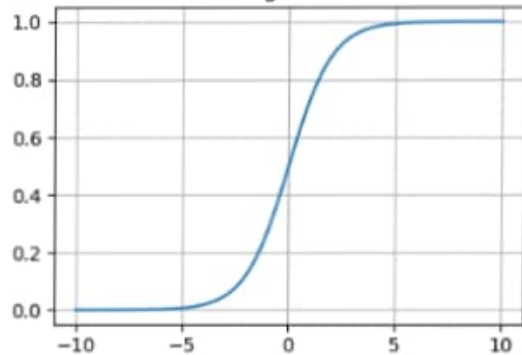
models = {
    "ReLU": NetReLU(),
    "Sigmoid": NetSigmoid(),
    "Tanh": NetTanh()
}

```

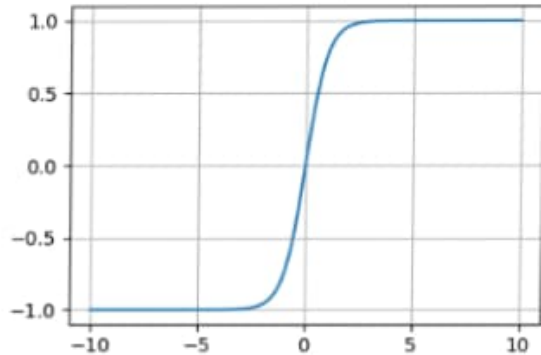
Linear



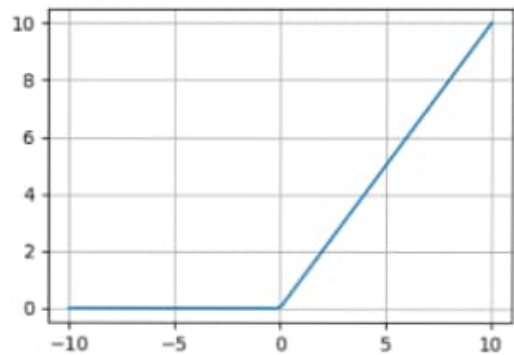
Sigmoid



Tanh



ReLU



Leaky ReLU

